Regenerative Autoencoders: Learning Spatial Structure in a Single Astronomical Image

Alejandro Hernandez Ruiz

First draft, version 0.1 — October 18, 2026

Abstract

Learning the appearance of small image patches does not determine how those patches are arranged. We investigate this distinction with Regenerative Autoencoders (ReGAE), a two-level architecture trained on a single large astronomical image. A patch encoder and conditional denoising decoder first learn local appearance. A second denoiser, called the Gencoder, uses the same architecture family to predict spatial windows of patch representations. Its target is encoded on a fractionally displaced image grid, so prediction changes the pixel support of every latent location rather than merely extending a border. Recurrent evaluation on overlapping windows provides a mechanism for generating and traversing larger image regions without global coordinate conditioning. We describe an implementation using 32×32 *RGB patches, 256-dimensional codes, and* 32×32 *latent windows, corresponding to approximately megapixel image regions. The case study uses a 236.8-megapixel James Webb Space Telescope mosaic of Pandora's Cluster. Archived training traces support optimization of both stages; historical qualitative observations motivate a discussion of coherent structure, noisy rendering, and transitions between visually ambiguous regions. We relate the construction to diffusion autoencoders, denoising dynamics, and hierarchical compression, while distinguishing these connections from established score-model or continuous-time equivalences. This exploratory report documents the architecture and existing evidence without adding new experiments.*

1. Introduction

The suggestion that a neural network "only memorizes" its training data leaves open a practical question: what must a model learn to reproduce a complex dataset, and where is that information stored? Even a single sufficiently large image contains structure at several scales. A model can reproduce local colors, textures, and objects while failing to recover their larger arrangement. Studying this gap provides a concrete way to separate a repertoire of appearances from a representation of their relationships.

We use a large image of Pandora's Cluster as such a setting. The task is visually simple to state: learn from the image and generate extended views that retain its spatial character. It is less simple to implement. A patch autoencoder stores information partly in shared network parameters and partly in the code assigned to each patch. Decoding the original codes in their original order supplies the arrangement externally. Sampling codes without that arrangement is a different problem. Familiar-looking patches alone do not identify which configurations belong together.

Regenerative Autoencoders (ReGAE) address this distinction through two learning stages. The first stage combines an encoder with a conditional denoising decoder. It learns to restore a clean 32×32 image patch from a corrupted input and its learned representation. The second stage learns a denoiser over spatial windows of these representations. We call this second network the *Gencoder*: it models how latent content is organized and how it changes under a small displacement of the image window. The decoder and Gencoder share an architecture family, with separately trained parameters and different channel dimensions.

The displacement is central to the construction. A shift by an integer number of latent cells leaves much of a window's code array directly reusable. Instead, we encode the image on several interleaved grids and train the Gencoder to predict the next grid. A displacement of approximately one quarter of an image patch changes the image support of every target code. Input and target still overlap strongly in pixel space, but the target requires predicting encodings of newly positioned crops throughout the window. Denoising and spatial prediction therefore form a single objective.

At generation time, the Gencoder repeatedly predicts overlapping windows of a latent canvas. Averaging their predictions combines multiple local views of the next state. The image decoder renders the resulting codes into RGB patches. Global coordinates in the original image are never supplied to the Gencoder. Any sustained traversal must arise from the content available in its current state. This creates an informative limitation: visually similar regions may not uniquely identify a continuation, allowing trajectories to depart from one source context and enter another.

This work extends the investigation of learned reconstruction dynamics in Neural Cellular Automata Manifold (NCAM) [6]. Here, regeneration means reconstruction under corruption, including reconstruction from noise when a patch code is available. The implementation uses multiscale networks and attention rather than retaining a strictly local cellular update rule. Its contribution is the combination of conditional patch regeneration, spatial latent prediction on displaced grids, and overlapping recurrent generation. We examine that combination as an exploratory single-image case study, with particular attention to what the code and preserved outputs substantiate.

The following sections place ReGAE beside denoising autoencoders, DiffAE, and single-image generative models; specify the implemented objectives and sampler; and discuss the existing evidence. A final discussion examines multiplicative gating, content-dependent traversal, architectural recursion, and the compression interpretation that connects these ideas. The aim is to preserve a concrete research direction with a precise account of its present scope.

2. Related Work

Regenerative neural dynamics. Growing Neural Cellular Automata learn local update rules that develop an image from a seed and can restore a damaged pattern [9]. Neural Cellular Automata Manifold (NCAM) extends this construction to an autoencoding setting, using dynamic convolutions to condition cellular development on an image representation [6]. ReGAE continues this investigation of learned reconstruction dynamics, with revised conditioning and a denoising objective. Its U-Net and attention components provide interactions beyond a strictly local cellular rule. The relevant continuity is therefore the use of learned, repeatedly applicable reconstruction operators, rather than an identical cellular architecture.

Denoising as generative modeling. Bengio et al. [2] show how alternating corruption and learned denoising can define a generative Markov chain, and introduce walkback training to address spurious modes encountered during sampling. Alain and Bengio [1] connect regularized reconstruction to the score of the data density, including the small-Gaussian-corruption regime. These results motivate interpreting denoisers as learning local structure around the data distribution. ReGAE uses a feature-space reconstruction loss, following the broader perceptual-loss approach of Johnson et al. [8], with VGG features [15], and an L_1 latent prediction loss. The score identities derived for particular reconstruction objectives do not directly establish a score interpretation for these losses.

Diffusion and continuous-time models. Denoising diffusion probabilistic models connect a prescribed forward noise process to a learned reverse process [7]. DDIM constructs non-Markovian processes with the same training objective and permits deterministic sampling [16]. Score-based SDEs unify several such constructions and yield a probability-flow ODE with matching time-marginal distributions under the corresponding score field [17]. Neural ODEs more generally parameterize a continuous-time derivative and compute trajectories using a differential-equation solver [3]. These perspectives suggest useful questions about ReGAE's trajectories, stability, and noise injection. The implementation studied here defines a discrete denoising map without explicit noise-time conditioning or a learned continuous-time velocity. Consequently, its recurrent application is a stochastic dynamical system, while identification with a reverse diffusion or probability-flow ODE would require additional assumptions and a derivation.

Autoencoding and latent generative structure. Diffusion Autoencoders (DiffAE) are a particularly close comparison [11]. They combine a learned semantic encoder with a conditional DDIM decoder, representing an image through a 512-dimensional nonspatial semantic vector and a spatial stochastic code. The latter retains image-specific variations needed for near-exact reconstruction; the semantic vector alone is not the complete code. DiffAE also learns a latent DDIM prior over semantic vectors for unconditional generation, using a deep multilayer perceptron with skip connections. Latent diffusion models similarly place generative denoising in the representation space of a pretrained autoencoder [13]. ReGAE's Gencoder learns relationships across a spatial array of patch representations and predicts the encodings of slightly displaced image patches. This changes the patch support throughout the target array and couples latent denoising to spatial prediction. Overlapping recurrent predictions then support image traversal without global coordinate conditioning. Our focus is this spatial transition mechanism and its integration with regenerative decoding, rather than the general presence of a second latent model.

Learning from a single image. SinGAN learns a hierarchy of patch distributions at different scales from one natural image, enabling synthesis that preserves appearance and larger structures [14]. It establishes the importance of multiscale internal image statistics for this setting. ReGAE studies a related division between local appearance and spatial organization through encoded patches and displaced latent prediction. The single-image setting makes the distinction between reconstructing a particular arrangement and generating configurations with familiar local content especially visible.

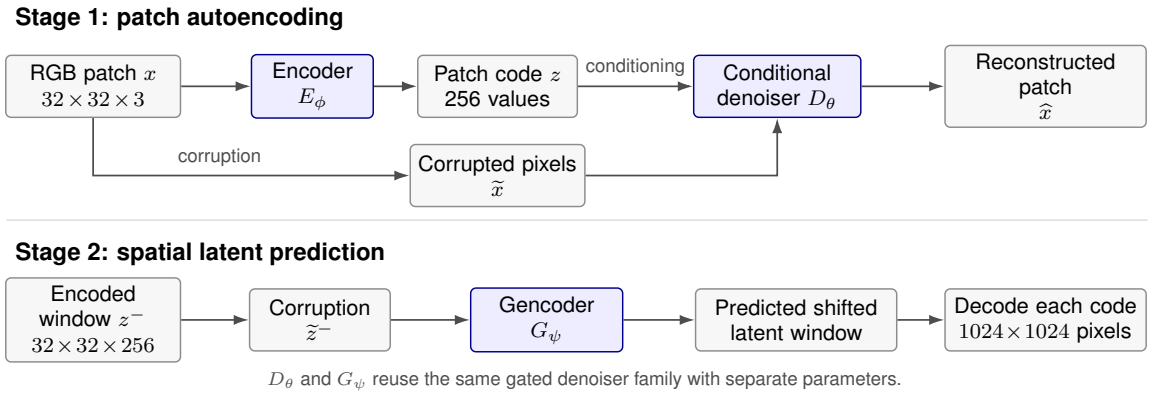


Figure 1. Two levels of regenerative modeling. The first stage learns a conditional image-patch decoder. The second learns a transition between spatial windows of patch codes, with targets taken from a fractionally shifted encoding grid. The image decoder and Gencoder use the same gated architecture family with separate parameters. During generation, overlapping Gencoder predictions update a latent canvas; each code is then rendered with fresh pixel noise.

Representation and compression. Minimum description length evaluates models through the length of their description together with the data description they support [12]. COIN provides an operational image-compression example by fitting a coordinate network and storing its quantized weights [5]. Recent work on language-model compression also connects predictive distributions to explicit coding procedures [4]. These perspectives distinguish a low-dimensional representation from a complete compressed bitstream. ReGAE's hierarchy concerns reusable descriptions of appearance and spatial structure; assessing it as a codec would additionally require accounting for precision, model storage, residual information, and reconstruction distortion.

3. Method

ReGAE separates local appearance from the spatial organization of local representations. An encoder E_ϕ and a conditional denoising decoder D_θ learn to reconstruct small image patches. A second network, the Gencoder G_ψ, learns a denoising transition between spatially displaced windows of their encodings. The same decoder architecture is instantiated at both levels, with separate parameters and different input channels. This section describes the archived v2 implementation; the recurrence belongs principally to latent-space sampling, rather than to an unrolled training objective.

3.1. Patch representation and denoising objective

Let $x \in [-1, 1]^{3\times p\times p}$ be a clean RGB patch, with $p = 32$. The encoder produces $z = E_\phi(x) \in \mathbb{R}^d$, where $d = 256$. Patches are sampled at random locations in the source image. A convolutional inlet is followed by four spatial downsampling stages, giving $256 \times 2 \times 2$ features. After flattening, two residual multilayer perceptron blocks map $1024 \to 256 \to 256$ to produce z. Its output is not constrained to unit norm, and training imposes no Gaussian-prior or vector-quantization objective.

We define the corruption operator

$$C_\alpha(x, \epsilon) = (1 - \alpha)x + \alpha\epsilon, \qquad \epsilon \sim \mathcal{N}(0, I). \tag{1}$$

The implementation samples k uniformly from $\{0, \dots, 7\}$ and u uniformly from $[0, 1)$, then sets $\alpha = f(k + u)$, where

$$f(s) = \mathrm{clip}_{[0,1]}\left(\frac{1.5^{-s} - \rho}{1 - \rho}\right), \qquad \rho = 0.0586. \tag{2}$$

Thus training includes nearly clean inputs and inputs dominated by noise. Within each batch, the second half reuses the first half's noise with opposite sign and the same corruption coefficients, paired with separately sampled clean patches. Neither k nor α is supplied explicitly to the network. This linear mixture is a corruption schedule, not an assertion of the variance-preserving forward process used in standard diffusion formulations.

The jointly optimized encoder and decoder minimize

$$\mathcal{L}_{\mathrm{AE}} = \mathbb{E}_{x,\alpha,\epsilon}\left[\ell_{\mathrm{perc}}\big(D_\theta(C_\alpha(x, \epsilon), E_\phi(x)), x\big)\right]. \tag{3}$$

Here $\ell_{\mathrm{perc}}(a, b)$ sums the mean absolute feature differences from four successive frozen VGG16 blocks. Inputs are mapped from $[-1, 1]$ to $[0, 1]$, normalized with the pretrained network's channel statistics, and bilinearly resized to 64×64. If Φ_ℓ includes this preprocessing and the first ℓ feature blocks, the loss is

$$\ell_{\mathrm{perc}}(a, b) = \sum_{\ell=1}^{4} \frac{\|\Phi_\ell(a) - \Phi_\ell(b)\|_1}{n_\ell}, \tag{4}$$

where n_ℓ is the number of feature entries. Each training example requires one decoder evaluation and predicts the

clean patch directly. There is no recurrent backpropagation through a sequence of regenerated images. At reconstruction time, a single call $D_\theta(\epsilon, z)$ can likewise decode a patch from its code and fresh Gaussian noise.

3.2. Conditional decoder and multiplicative gate

The decoder contains a convolutional inlet, a U-shaped network with group normalization, a multiplicative feature gate, a second U-shaped network without normalization, and an output projection. Multiscale skip connections, learned patch-relative positional tensors, and attention within the processed window provide spatial interaction. Latent-dependent transformations modulate intermediate features through learned additive and multiplicative terms. Consequently, the update operates over an entire patch or latent window rather than being restricted to a fixed local cellular neighborhood.

The first U-shaped network produces two feature tensors, h and g. The v2 gate, informally called *gravity*, computes

$$\widetilde{h} = h \odot \gamma(g), \qquad \gamma(g) = \mathrm{clip}_{[0,2]}(1 + g/3) + 10^{-4}. \quad (5)$$

Its multiplier therefore lies in $[10^{-4}, 2 + 10^{-4}]$. The earlier v0 decoder instead used $\widetilde{h} = h \odot \exp(g)$. The replacement bounds this particular gain; it does not impose a global activation bound or prove stability under repeated updates. Subsequent transformations can still amplify features. Both D_θ and the v2 Gencoder use this gated architecture, but the latter receives no separate conditioning vector.

3.3. Shifted latent grids

Let the source image have height H and width W, and define $H_z = \lfloor H/p \rfloor$ and $W_z = \lfloor W/p \rfloor$. With the first stage fixed, we construct $K = 4$ offset grids $Z^{(o)} \in \mathbb{R}^{d \times H_z \times W_z}$, $o \in \{0, \ldots, K-1\}$. To match the actual crop helper, their pixel origins are

$$r_o(i) = \left\lfloor \frac{(i + o/K)(H - p)}{H_z} \right\rfloor, \quad (6)$$

$$c_o(j) = \left\lfloor \frac{(j + o/K)(W - p)}{W_z} \right\rfloor. \quad (7)$$

The entry $Z^{(o)}_{:,i,j}$ encodes the $p \times p$ crop starting there. Nonzero-offset maps omit their final row and column during construction; training windows are selected to avoid these unfilled entries. This convention approximates a regular patch lattice but is not exactly $r_o(i) = pi + po/K$, owing to image dimensions and integer rounding.

For a $q \times q$ window with $q = 32$, sample an offset o and a valid origin (i, j). Set $o' = (o + 1) \bmod K$ and $b = \lfloor (o + 1)/K \rfloor$. The paired windows are

$$z^- = Z^{(o)}_{:,i:i+q,\, j:j+q}, \quad (8)$$

$$z^+ = Z^{(o')}_{:,i+b:i+b+q,\, j+b:j+b+q}. \quad (9)$$

The wrap at $o = K - 1$ advances the integer lattice origin, maintaining a positive displacement in both source-image axes. Each target code describes a shifted pixel patch, rather than copying the overlapping input tokens and predicting only a newly exposed border. The displacement is approximately $p/K = 8$ pixels per axis, with rounding-dependent variation.

The Gencoder minimizes the elementwise mean absolute error

$$\mathcal{L}_G = \mathbb{E}\left[\frac{\| G_\psi(C_\alpha(z^-, \epsilon)) - z^+ \|_1}{dq^2} \right], \quad (10)$$

using the same corruption schedule and sign-paired noise construction. Encoder and image-decoder parameters are not optimized in this stage. A latent window decodes into $pq \times pq = 1024 \times 1024$ pixels. Its source-image support instead spans $r_o(i+q-1) - r_o(i) + p$ rows and the analogous number of columns: the rounded grid makes this slightly smaller than the decoded extent. Global source-image coordinates and the offset index are absent from the Gencoder's inputs, although its internal positional tensors distinguish locations within a window. Spatial progression must therefore be inferred from the available content.

3.4. Optimization

Both stages use Adam with AMSGrad, an initial learning rate of 5×10^{-5}, moment coefficients $(0.5, 0.9)$, numerical stabilizer 10^{-8}, and no weight decay. The nominal batch contains 64 examples. The first stage is configured for 256 epochs, halving its learning rate every 64 epochs and increasing gradient accumulation from one to two batches at epoch 128. The second stage is configured for 1,024 epochs, increasing accumulation from one through four batches at 256-epoch intervals. Accumulated losses are divided by the number of contributing batches before backpropagation.

An epoch is a bookkeeping interval, not an exhaustive pass over disjoint crops. Both stages define its length as $\lfloor H_z W_z / 64 \rfloor$, or 3,607 updates for the recorded image dimensions, while drawing training locations randomly. In particular, a Gencoder epoch is not normalized by the number of latent sites in each large window. The frozen VGG features provide the first stage's perceptual objective; they are not an additional trainable branch. No multi-step rollout loss couples successive sampled transitions during optimization.

3.5. Overlapping recurrent generation

The final video routine maintains a latent canvas $S_t \in \mathbb{R}^{d \times A \times B}$ with $A = \lceil 1080/p \rceil = 34$ and $B = \lceil 1920/p \rceil = 60$. It initializes S_0 with Gaussian noise. Its optional seven-step initialization loop is commented out; generation begins directly with the recurrent update below.

Let $\mathcal{P}$ be the multiset of $q \times q$ window origins produced by integer-rounded evenly spaced coordinates. In this configura-

tion the row sequence is $(0, 0, 1, 2)$ and the column sequence is $(0, 9, 18, 28)$. Repeated origins are retained. Write W_s for window extraction and $W_s^\top$ for placing a window into a zero canvas. The coverage count is $N = \sum_{s\in\mathcal{P}} W_s^\top \mathbf{1}$, broadcast across latent channels.

Noise refresh is stronger near the bottom and right edges. For canvas coordinates (r, c), define

$$\delta(r, c) = \min(A - 1 - r, B - 1 - c), \tag{11}$$
$$m(r, c) = \min(6, 4 + \delta(r, c)), \tag{12}$$
$$a_t(r, c) = (1 + \beta_t) f(m(r, c)), \tag{13}$$
$$\beta_t = \tfrac{1}{2}\left[1 + \cos(t/1824)\right]. \tag{14}$$

The denominator is the configured 114 seconds times 16 frames per second; it is the cosine's argument scale, not a full-period duration. Extending C to elementwise masks, the synchronous update is

$$S_{t+1} = N^{-1} \odot \sum_{s\in\mathcal{P}} W_s^\top G_\psi(C_{W_s a_t}(W_s S_t, \eta_{t,s})), \tag{15}$$

with independent Gaussian $\eta_{t,s}$ for each window evaluation. Every prediction reads the previous canvas; an initially zero accumulator collects predictions before normalization by coverage. Earlier exploratory routines also used overwrite-based overlap handling, but Eq. (15) describes the final averaging routine.

There is no explicit spatial roll, coordinate update, or translated copy in this loop. Motion is learned through Eq. (9). If the model tracks a consistent source region, advancing the viewed crop down and right corresponds to apparent image-content motion up and left. That interpretation is conditional on coherent tracking, not a guaranteed trajectory.

Each displayed frame is then generated independently in RGB space. For every latent location v, the routine draws fresh pixel noise $\xi_{t,v}$ and computes

$$\widehat{x}_{t,v} = Q(D_\theta(\xi_{t,v}, S_{t+1,:,v})), \tag{16}$$

where Q clips to $[-1, 1]$ and quantizes to 256 levels. Nonoverlapping decoded patches form a 1088×1920 image, centrally cropped to 1080×1920. The previous RGB frame is never fed back: latent refresh and fresh decoder noise are separate stochastic sources. The configured output is 7,280 frames at 16 frames per second, or 7 minutes 35 seconds; this specifies the routine, not evidence that its saved progress widget records completion.

Implementation qualifications, including the historical learning-rate schedule and earlier sampling cells, are documented in Appendix A.

Quantity	Image AE	Gencoder
Trainable parameters	55.66 M	85.48 M
Configured epochs	256	1,024
Displayed minimum	0.389	0.235
Displayed current value	0.450	0.252

Table 1. Archived training summaries. Displayed losses are the square root of the corresponding batch objective: a VGG feature L_1 objective for the AE and a latent L_1 objective for the Gencoder. They are neither RMSE values nor directly comparable between columns. Epochs describe the configured training loops.

4. Case Study and Existing Evidence

4.1. A single large image

The archived v2 run loads a rendered RGB mosaic of Pandora's Cluster from the James Webb Space Telescope's NIRCam instrument [10]. Its recorded tensor dimensions are $3\times13{,}422\times17{,}644$, totaling 236,817,768 pixels. The source release dates to February 2023. This is a display composite, not a collection of independent photographs or a raw photometric dataset. We treat it as one spatially structured training image.

The stored configuration produces 419×551 latent sites per encoding grid and four interleaved grids for displaced prediction. The image autoencoder has 55,660,048 trainable parameters, of which 4,673,752 belong to the encoder. The Gencoder has 85,479,680 parameters. The pretrained VGG network supplies a fixed perceptual objective and brings information learned outside this image; it is excluded from these trainable-parameter counts. The experiment therefore concerns training the ReGAE networks on one image, rather than learning every component from scratch using that image alone.

4.2. Preserved optimization traces

The notebooks preserve loss plots for both stages, reproduced in Appendix B. Both exhibit a rapid initial decrease followed by a slower approach to a plateau. Table 1 transcribes their displayed summaries. These are training traces under random crop sampling; they do not report held-out reconstruction or generation performance.

The plots' horizontal axes are labeled "epoch" by the historical plotting helper, but advance per optimizer update. An epoch in the training loop comprises 3,607 such updates. The AE plot also clips its displayed history to $[0, 1]$. Raw loss arrays, independent evaluation metrics, and hardware or runtime measurements are not included in the archive. The traces support the limited conclusion that the specified objectives were optimized over extended training; they do not isolate the contribution of the shift, gate, or overlap strategy.

4.3. Sampling behavior reported during development

During development, local patch sampling produced starfield appearance without reliably reproducing the source image's larger arrangement. This motivated learning a spatial model over the patch codes. The author reports that the later Gencoder produced larger coherent regions resembling the source, together with stochastic variations and a persistent noisy visual quality compared informally to an old VHS recording. These observations concern existing runs; they have not been re-created for this manuscript.

The author also reports occasional abrupt changes in the region being depicted, described as "teleportation." The proposed explanation is ambiguity in sparse or repeated-looking content: without global coordinates, a state compatible with several source contexts can evolve toward another familiar configuration. This is a plausible interpretation of a content-conditioned transition, but the archive does not contain source-location tracking or a quantitative test of this explanation. In particular, a plotted latent mean-square value is not an entropy estimate and cannot establish an entropy-based cause.

The final sampler provides two identifiable sources of visual variability: repeated noise injection in latent space and fresh image-decoder noise for every rendered patch and frame. These mechanisms can alter successive frames even if the latent trajectory remains nearby. The present evidence does not separate their effects from approximation error, patch boundaries, or overlap averaging. The routine is configured to produce a 1080×1920 video at 16 frames per second; the preserved progress display alone does not document a completed render.

Status of visual evidence. The generated stills, videos, and checkpoints described above have not yet been incorporated into this first draft. They are therefore reported as historical qualitative observations, not presented as independently inspectable results. The manuscript currently includes the architecture and original training plots. Adding representative frames and short trajectory excerpts from the existing runs is the next documentation step and does not require a new experiment.

5. Discussion

5.1. Regeneration, dynamics, and ambiguous continuations

The Gencoder is trained to map a corrupted observation of one spatial window to a clean observation of a nearby window. It is consequently both a restoration operator and a predictor. With elementwise L_1 training, an optimal unconstrained predictor would return coordinatewise medians of z^+ conditional on the corrupted input. This observation helps explain why one-step regression need not reproduce the full distribution of possible continuations. Injected noise makes the sampled transition stochastic, but does not by itself make its transition probabilities equal to those of the source image.

Writing the full canvas update as

$$S_{t+1} = F_\psi(S_t, \eta_t; t) \tag{17}$$

makes its dynamical character explicit. The explicit t dependence comes from the refresh schedule, and η_t collects the noise used across windows. Setting the noise draws to zero defines a formal displacement field $F_\psi(S, 0; t) - S$; for a nonlinear network, this generally differs from the expected displacement under random noise. Treating either field as a continuous-time derivative would define a new model: the implementation neither trains a small-step limit nor uses an ODE integrator. Similarly, the losses in Eqs. (3) and (10) do not establish the score needed for a probability-flow ODE [17]. The connection to diffusion is the use of learned denoising and latent generative structure, with materially different objectives and sampling rules.

The absence of global coordinates makes continuation depend on recognizable content. A unique local configuration can provide a strong cue to the next view; several similar configurations can support competing continuations. This offers a useful interpretation of reported jumps without requiring the network to maintain an explicit location variable. Such jumps could also arise from accumulated prediction error or departure from the training distribution. The current observations cannot distinguish these mechanisms.

5.2. Multiplicative gain and the "gravity" analogy

The gravity branch scales a feature field pointwise. Multipliers above one amplify a feature and those below one attenuate it; spatial interactions elsewhere in the network determine how this affects an image. The analogy is an intuitive name for learned gain, not a physical gravitational law or an explicit transport of mass between pixels.

The unbounded exponential gate in v0 motivated a memorable development observation. The author recalls bright stars becoming progressively brighter, followed by large dark regions, termed "neural black holes." The relevant checkpoints and recordings were not retained, so the event is an anecdote rather than a reproducible result. Exponential gain offers a possible mechanism for amplification: in an isolated gate-only recurrence $h_{t+1} = h_t \exp(g_t)$, one obtains $h_T = h_0 \exp(\sum_{t<T} g_t)$. This toy calculation does not model the remaining transformations or identify the cause of the reported darkening. In particular, the code supplies no verified overflow-triggered reset mechanism. The bounded v2 gate removes unbounded growth from this individual operation, while leaving stability of the complete recurrence unresolved.

Representation or parameter payload	MB
Source RGB array, 8 bits per channel	710.45
One $419 \times 551 \times 256$ grid, FP32	236.41
Image encoder and decoder, FP32	222.64
Gencoder, FP32	341.92
Decoder plus one latent grid, FP32	440.36
AE and Gencoder parameters, FP32	564.56

Table 2. Uncompressed payload accounting. Rows are different accounting choices, not interchangeable complete codecs. Optimizer state and container overhead are excluded. Four shifted training grids occupy approximately 945.64 MB in FP32.

5.3. Architectural reuse across levels

Reusing the decoder family for latent prediction expresses a simple hypothesis: a useful reconstruction operator for observable fields may also model fields of learned representations. The present hierarchy instantiates this idea at two levels. The networks have different parameters, dimensions, and conditioning, and the higher level models encoder outputs rather than the lower network's parameter dynamics.

This is a motivation for recursively extensible architectures, not a demonstration that a network has learned its own learning algorithm. No task distribution, meta-optimization objective, or self-modifying training procedure is evaluated here. Further levels could be defined by repeating the representation-and-prediction construction, but their effectiveness is untested. The value of the current experiment is more concrete: the same building blocks can express both image reconstruction and spatial prediction among image codes.

5.4. What is compressed?

A 32×32 RGB patch contains 3,072 scalar channel values and is represented by 256 latent scalars: a factor of 12 in scalar count. At 8 bits per RGB channel and 32 bits per latent scalar, the nominal payload reduction is instead a factor of three, before storing a decoder or accounting for reconstruction distortion. Numerical precision is therefore part of the argument, not a secondary implementation detail.

Table 2 separates several storage quantities using decimal megabytes. These are calculations from dimensions and parameter counts, not measurements of compressed files. The actual crop lattice also has boundary and rounding conventions, so a single code grid is not itself a specified exact codec for every source pixel.

The hierarchy suggests a useful conceptual division. The first stage shares a model of appearance across many patches; a spatial code grid then specifies where those appearances occur. The second stage learns regularities in that grid. It can therefore reduce the amount of spatial information that must be supplied to generate plausible content. It does not follow that it has replaced the exact grid by a lossless description: recovering a particular source arrangement could still require an initial state, a trajectory specification, and residual information.

A minimum-description-length account would include both model and data terms [12]. For a practical lossy codec, the relevant object is a bitstream together with a reconstruction distortion, including the cost of any non-shared decoder. ReGAE specifies neither entropy coding nor quantized model storage nor a rate–distortion evaluation. The reported large checkpoint sizes may also include optimizer tensors: AMSGrad stores three moment-related tensors per parameter in addition to the weights. This can bring FP32 training checkpoints near the author's recalled 0.89 GB and 1.4 GB without implying that inference requires that storage. A broader claim that intelligence is compression remains an interpretive position; this experiment provides a concrete example of progressively modeling local and relational information, rather than a test of that claim.

5.5. Limitations and next documentation steps

The study uses one image and lacks controlled ablations, held-out evaluation, and comparisons at matched capacity or compute. Its fixed perceptual objective draws on externally pretrained features. One-step training does not explicitly optimize long-horizon behavior or establish recurrent stability, and uncertainty is not calibrated. Historical notebook inconsistencies also limit immediate end-to-end reproduction. These boundaries are part of the scope of this first report.

The immediate next steps are documentary: attach existing generated examples, identify their checkpoints and sampler settings, and align descriptions with the author's records. The related Neural Vapor Deposition work concerns progressive parameter growth and removal and is reserved for a separate manuscript. Neither that study nor new ReGAE training is needed to state the method presented here. Quantitative evaluation and larger recursive hierarchies remain possible future research directions.

6. Conclusion

ReGAE separates learning local appearance from learning the spatial organization of the resulting representations. Its central mechanism is a denoising predictor over spatial latent windows whose targets correspond to fractionally displaced image patches. Reusing the denoiser architecture across levels supports a simple hierarchical construction, and overlapping recurrent sampling turns the learned transition into a procedure for image generation and traversal. The archived implementation and training traces establish a concrete starting point for studying this construction. Its reported long-range visual behavior remains qualitative, and neither faithful navigation nor stable recurrence follows from the one-step objectives alone. The broader opportunity is

to study how reusable reconstruction operators can organize information at successively larger scales, with explicit attention to the distinction between reconstructing stored structure and generating plausible alternatives.

Data and code. The original notebooks and manuscript sources are available in the Neural Pandora's Cluster repository. The source image is credited to NASA, ESA, CSA, Ivo Labbe (Swinburne), and Rachel Bezanson (University of Pittsburgh), with image processing by Alyssa Pagan (STScI) [10]. The experiments use the rendered image as a visual learning problem; generated outputs are not astronomical observations.

References

[1] Guillaume Alain and Yoshua Bengio. What regularized auto-encoders learn from the data-generating distribution. *Journal of Machine Learning Research*, 15(110):3743–3773, 2014. 2

[2] Yoshua Bengio, Li Yao, Guillaume Alain, and Pascal Vincent. Generalized denoising auto-encoders as generative models. In *Advances in Neural Information Processing Systems*, 2013. 2

[3] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems*, 2018. 2

[4] Grégoire Delétang, Anian Ruoss, Paul-Ambroise Duquenne, Elliot Catt, Tim Genewein, Christopher Mattern, Jordi Grau-Moya, Li Kevin Wenliang, Matthew Aitchison, Laurent Orseau, Marcus Hutter, and Joel Veness. Language modeling is compression. In *International Conference on Learning Representations*, 2024. 3

[5] Emilien Dupont, Adam Goliński, Milad Alizadeh, Yee Whye Teh, and Arnaud Doucet. COIN: COmpression with Implicit Neural representations. *arXiv preprint arXiv:2103.03123*, 2021. 3

[6] Alejandro Hernandez, Armand Vilalta, and Francesc Moreno-Noguer. Neural cellular automata manifold. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10020–10028, 2021. 2

[7] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, 2020. 2

[8] Justin Johnson, Alexandre Alahi, and Li Fei-Fei. Perceptual losses for real-time style transfer and super-resolution. In *European Conference on Computer Vision*, 2016. 2

[9] Alexander Mordvintsev, Ettore Randazzo, Eyvind Niklasson, and Michael Levin. Growing neural cellular automata. *Distill*, 2020. 2

[10] NASA. Pandora's cluster (NIRCam image). NASA Science, 2023. Released February 15, 2023. 5, 8

[11] Konpat Preechakul, Nattanat Chatthee, Suttisak Wizadwongsa, and Supasorn Suwajanakorn. Diffusion autoencoders: Toward a meaningful and decodable representation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022. 2

[12] Jorma Rissanen. Modeling by shortest data description. *Automatica*, 1978. 3, 7

[13] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022. 2

[14] Tamar Rott Shaham, Tali Dekel, and Tomer Michaeli. SinGAN: Learning a generative model from a single natural image. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019. 2

[15] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations*, 2015. 2

[16] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations*, 2021. 2

[17] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2021. 2, 6

A. Archive and Implementation Details

Source of the technical description. The principal implementation is the v2 notebook, while v0 documents architectural and sampling development. The repository preserves both original files in its notebooks directory, together with their SHA-256 hashes. The description in this manuscript was obtained by inspecting code and saved outputs. No notebook training or generation cell was executed to prepare this draft. The separate NVD notebooks are not evidence for the ReGAE results.

Stage boundaries. The v2 configuration jointly trains the image encoder and decoder, then constructs latent maps for the Gencoder. The latter is a separately trained instance of the decoder family with 256 input/output channels and no external conditioning code. The image decoder uses three input/output channels and latent conditioning. Shared architecture thus refers to the computational construction, not tied weights. The patch-relative positional parameters do not encode global source-image coordinates.

Optimization and reproducibility. The recorded environment reports PyTorch 2.2.2 with CUDA 12.1, and the code sets a random seed of 4202. These facts do not specify the original GPU, elapsed training time, or complete software environment. The archive contains configured training loops and saved displays, but no complete execution log establishing every historical state. In v2, the Gencoder loop computes a decaying learning rate but assigns it to the AE optimizer, leaving the Gencoder optimizer unchanged by that schedule. This is preserved as a historical qualification rather than silently corrected. Exact reproduction should distinguish a faithful historical configuration from a cleaned implementation.

Exploratory sampling in v0. The earlier notebook includes source-code reconstruction, latent generation, random overlapping patch updates, repeated local refinement, and shifted raster sweeps. Some updates protect previously completed pixels or use update-budget masks. Their motivation includes an analogy to wave-function-collapse-style synthesis. The cells do not implement a documented entropy-ranked constraint-propagation algorithm, and their current channel and conditioning assumptions are not mutually consistent with all retained network definitions. We therefore avoid presenting them as a verified benchmark of sampling strategies. The later v2 synchronous overlap average is specified explicitly in Eq. (15).

Extent of a latent window. The decoder tiles 32×32 generated image patches into a 1024×1024 macro-image. The source crops used to build a corresponding training window are positioned by the rounded lattice in Eq. (7), rather than exactly 32 pixels apart. "Megapixel window" consequently describes the decoded tile extent; it does not assert that all sampled training windows form an exact, gap-free partition of the source image.

Visual evidence to add from existing runs. The most informative additions would be a source region alongside its patch-code reconstruction, a spatially coherent Gencoder sample, and a short sequence showing a typical trajectory and a reported jump. Each should identify its checkpoint, sampler configuration, and whether its RGB rendering uses fresh noise. The unstable exponential-gate example cannot currently be illustrated from an original recording and should remain identified as an anecdote. This list specifies documentation needs, not new experiments or figures that already exist in this draft.

B. Preserved Training Plots

The following plots are extracted directly from the saved PNG outputs of `main_v2.ipynb`, without redrawing curves or changing image content. They are included to preserve the evidence available in the working notebook. Their labels inherit the historical plotting helper's conventions: the horizontal count is optimizer updates despite the printed "epoch" label, and the vertical quantity is the square root of a batch objective, not a pixel-space error metric. The two losses use different feature spaces and are not comparable in absolute magnitude.

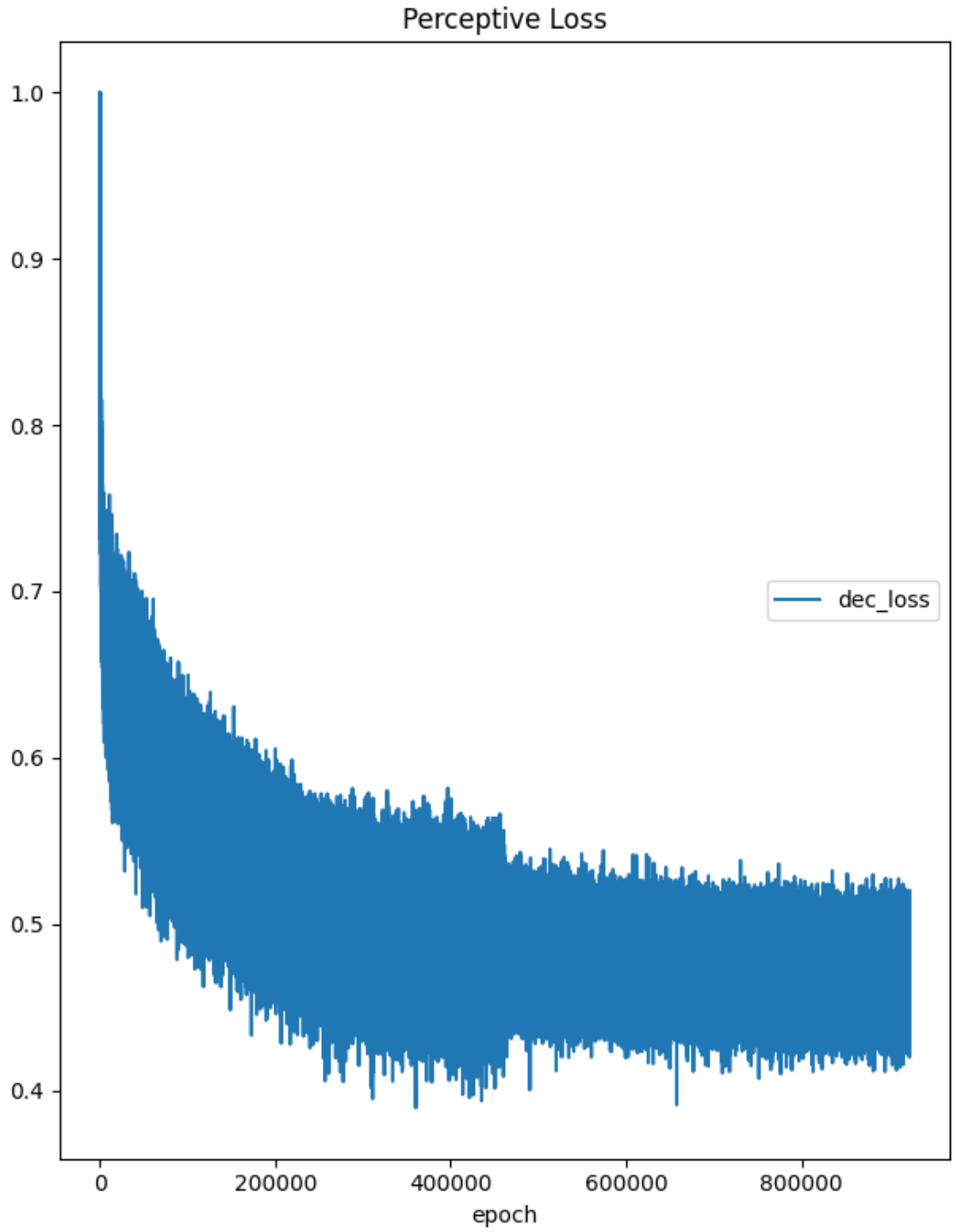


Figure 2. Saved image-autoencoder training display (v2, code cell 39 under zero-based notebook indexing). The summary reports minimum 0.389, current 0.450, and maximum 1.000; the history was clipped to $[0, 1]$ for plotting. The displayed quantity is the square root of the VGG feature L_1 loss. The history supports optimization of the training objective, not a held-out quality estimate.

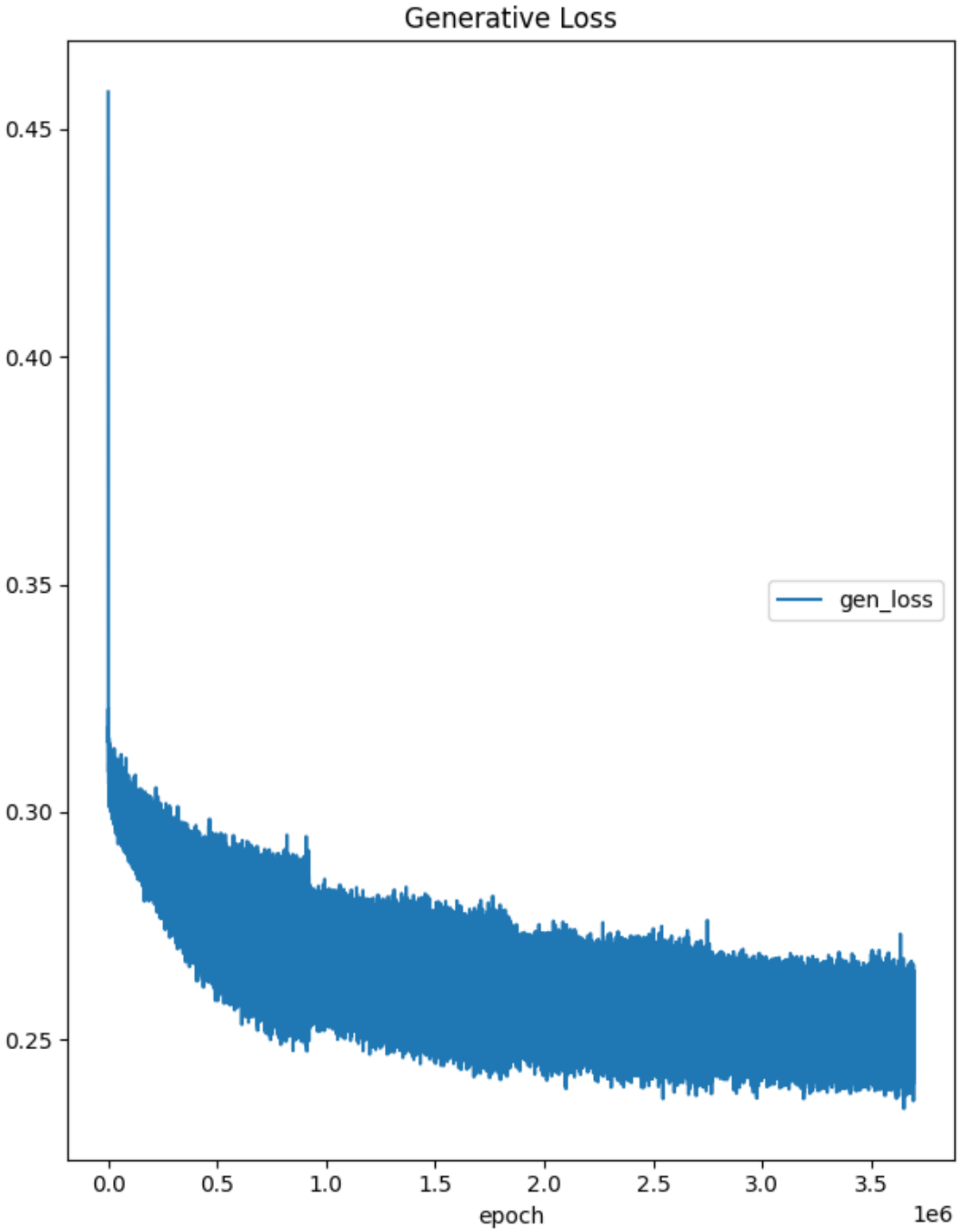


Figure 3. Saved Gencoder training display (v2, code cell 46 under zero-based notebook indexing). The summary reports minimum 0.235, current 0.252, and maximum 0.458. The displayed quantity is the square root of the latent L_1 prediction loss. No new training was run to produce this figure.